

Sikizwa Voice 360° System Report

Optimized tool selection! I'll create a short task plan, then inspect key source files to build the report.

Created 5 todos

Now I'll read the main backend, AI service, mobile app, and admin dashboard files to extract specifics.

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/backend/api-server/src/app.js]]`

Progress update: I've loaded the backend app.js. Next I'll read the AI service core and its processing modules. Listing the AI service `app` directory to find available files.

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/ai-service/app]]`

I'll list the `ai-service` directory to confirm what's present.

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/ai-service]]`

Reading the AI service `main.py` and service modules.

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/apps/ai-service/app/main.py]]`

I'll read the AI service modules: `transcribe.py`, `emotion.py`, `risk.py`, `chat.py`.

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/apps/ai-service/app/services/transcribe.py]]`

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/apps/ai-service/app/services/emotion.py]]`

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/apps/ai-service/app/services/risk.py]]`

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/apps/ai-service/app/services/chat.py]]`

Now I'll read the mobile app entry point to summarize client behavior.

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/apps/mobile-app/lib/main.dart]]`

I'll open the admin dashboard `App.tsx` to capture front-end flow.

Read `[[file:///c%3A/Users/SHARON%20Otunga/Desktop/Sikizwa%20Voice%20360%C2%B0/sikizwa-system/apps/admin-dashboard/src/App.tsx]]`

Updating the task plan: mark inspection done and start drafting architecture summary.

Starting: *Summarize architecture and data flow* (3/5)

Progress: I created a short task plan and inspected the backend, AI service, mobile app, and admin dashboard source files. Next: architecture summary and full report.

System Report — Sikizwa Voice 360°

Overview

- **Purpose:** Sikizwa Voice 360° is a safety & emotional-wellness platform that captures user voice input (mobile + pendant devices), analyzes audio with an AI microservice (transcription, emotion, risk scoring, conversational support), and surfaces results to admins via a web dashboard and to backend workflows (reports, notifications, emergencies).
- **Top-level structure:** Root workspace contains the system documentation and these primary apps:
 - Backend API server: `backend/api-server/src/app.js`
 - AI microservice: `apps/ai-service/app/main.py`
 - Mobile app (Flutter): `apps/mobile-app/lib/main.dart`
 - Admin dashboard (React + Vite): `apps/admin-dashboard/src/App.tsx`
 - Infrastructure manifests under `infrastructure/` and top-level docker-compose files.

Components & Responsibilities

- **Mobile App (`apps/mobile-app`)**
 - Entry: `apps/mobile-app/lib/main.dart`
 - Behavior:
 - Initializes secure storage, API service, session manager and device pairing checks.
 - Determines initial route: authenticated (`/home`), pairing (`/pairing/generate`), or login.
 - Uses `ApiService` for backend communication and secure session handling (cookies / sessions).
 - Role: capture audio, pairing with pendant devices, user interactions, and send data to backend.
- **Backend API Server (`backend/api-server`)**
 - Entry: `backend/api-server/src/app.js`
 - Architecture:
 - Express.js app with dual routing model:
 - `/web/*` — stateful routes for browser UI: session + CSRF protection (e.g., `/web/csrf-token`, `/web/docs`, `/web/admin`).
 - `/api/*` — stateless JSON APIs for mobile and services (no session/csrf).
 - Middleware: `helmet`, `cors` (configurable origins), `cookie-parser`, `xss-clean`, `morgan` logging, rate limiting (`express-rate-limit`), session via `express-session` for `/web`.
 - Routes mounted (not exhaustive): auth, admin, reports, emergencies, counsellors, ai, uploads, notifications, supportChats, learningResources, safeSpaces, analytics, distressSignals, device, user.
 - Health endpoint at `/health`.
 - Key security features:
 - Rate limiting with separate general and auth limiters.
 - CSRF protection for web routes only.
 - CORS origin whitelist plus optional `CORS_ORIGINS` env override.
 - Logging of requests and rate-limit events.
 - Error handling: central `ApiError` + custom error handler.
- **AI Service (`apps/ai-service`)**
 - Entry: `apps/ai-service/app/main.py`
 - Exposes FastAPI endpoints:
 - `POST /transcribe` — accepts audio file, returns transcription (uses OpenAI audio transcription; `WHISPER_MODEL`).
 - `POST /emotion-analysis` — returns emotion probabilities and derived `risk_level`.
 - `POST /risk-score` — accepts text payload, returns risk score & level.
 - `POST /chat` — conversational reply from OpenAI-based chat model with an empathetic system prompt.

- Internals (modules under `app/services`):
- transcribe.py: saves upload, calls OpenAI Audio Transcription, cleans temporary file.
- Reference: apps/ai-service/app/services/transcribe.py
- emotion.py: uses `transformers` audio-classification pipeline (default model `superb/hubert-large-superb-er`) and `librosa` to compute energy and emotion labels → urgency mapping.
- Reference: apps/ai-service/app/services/emotion.py
- risk.py: keyword heuristic + text-classification pipeline to compute numeric risk score and categorical level.
- Reference: apps/ai-service/app/services/risk.py
- chat.py: wraps OpenAI chat completions with a fixed empathetic system prompt and fallback reply.
- Reference: apps/ai-service/app/services/chat.py
- Operational notes:
- Service warms up emotion & risk pipelines at startup.
- Requires env vars: `OPENAI\_API\_KEY`, `WHISPER\_MODEL` (and optional model envs for emotion/risk/chat).
- CORS is permissive (`allow\_origins=["\*"]`) by default; production deployments should restrict origins.

****Admin Dashboard (`apps/admin-dashboard`)****

- Entry: apps/admin-dashboard/src/App.tsx
- Behavior:
- Authenticated admin experience using secure httpOnly cookies (backend session).
- Routes: dashboard, reports, emergencies, counsellors, login/signup.
- Uses `fetchAdminProfile()` to detect session; sign-out uses `adminLogout()`.
- Role: real-time safety monitoring, managing counsellors, viewing reports and emergencies.

****Data Flow (typical scenarios)****

- Voice capture → Mobile app:
- User records audio on mobile (or a pendant triggers capture).
- Mobile app uploads audio to backend `/api/uploads` or directly to AI service (system uses backend `/api/ai` route).
- Backend → AI Service:
- Backend AI route (`/api/ai`) proxies or orchestrates calls to the AI microservice endpoints (`/transcribe`, `/emotion-analysis`, `/risk-score`, `/chat`); the backend may persist results, trigger notifications, or escalate emergencies.
- Risk/emergency escalation:
- If `risk\_score` or emotion urgency reaches thresholds, backend triggers emergency flows: create emergency report, notify counsellors, fire push/real-time notifications, or call external emergency integrations.
- Admin actions:
- Admin views reports and can manage users/counsellors; uses session-authenticated UI under `/web` or `/admin` routes.

****APIs & Endpoints (high-level)****

- Backend (examples from app.js):
- `/api/auth/\*` — authentication
- `/api/reports/\*` — reports CRUD
- `/api/emergencies/\*` — emergency endpoints
- `/api/ai/\*` — AI orchestration endpoints
- `/health` — health check
- `/web/csrf-token` — CSRF token for browser UI
- (See `backend/api-server/src/app.js` for full route mounting.)
- AI Service:
- `POST /transcribe` → { text }

- ``POST /emotion-analysis`` → { emotions, risk_level }
- ``POST /risk-score`` → { score, level, emotion_tags }
- ``POST /chat`` → { reply }
- (See `apps/ai-service/app/main.py`)

****Tech Stack****

- Backend: Node.js + Express, security middleware (helmet, csrf), logging, rate-limiting.
- AI microservice: Python, FastAPI, Hugging Face ``transformers``, ``librosa``, OpenAI SDK for Whisper/Chat.
- Mobile: Flutter (Dart), Riverpod state management.
- Admin frontend: React + Vite + TypeScript (Tailwind CSS config present).
- Containerization / Infrastructure: Dockerfiles and docker-compose manifests under ``infrastructure/`` and per-app Dockerfiles (see ``apps/*/`` Dockerfiles and ``infrastructure/docker/docker-compose.yml``).

****Security & Hardening****

- Server-side:
 - Helmet for HTTP headers.
 - CORS origin whitelist plus env override.
 - XSS cleaning via ``xss-clean``.
 - Rate limiting with separate auth limiter.
 - CSRF protection and session cookies scoped to ``/web`` browser flows.
 - Logging of requests, rate-limit events, and suspicious conditions.
- AI & Data:
 - Temporary audio files are cleaned after processing (see `transcribe.py` and `emotion.py`).
 - AI keys and models are expected in env vars; secure secrets management required in production.
 - Recommended improvements:
 - Restrict AI service CORS in production.
 - Ensure all inter-service traffic is authenticated (mutual TLS or service tokens).
 - Centralized audit logs and rotation for PII-sensitive events.
 - Rate-limit or sanitize large uploads on AI endpoints.

****Deployment & Running****

- Each app has a Dockerfile / manifests; root and ``infrastructure/`` contain compose files for local orchestration.
- Env vars to check for AI service: ``OPENAI_API_KEY``, ``WHISPER_MODEL``, ``EMOTION_MODEL``, ``RISK_MODEL``, ``OPENAI_CHAT_MODEL``.
- Backend environment: ``NODE_ENV``, ``SESSION_SECRET``, ``CORS_ORIGINS``, ``SESSION_NAME``, etc.
- Health checks:
 - Backend: ``GET /health``
 - AI service: ``GET /health``
- Quick local dev (examples — run from each app folder):
 - Backend (node):
 - `npm install`; run via ``npm run dev`` or node process configured in ``package.json``.
 - AI service (python):
 - create virtualenv, install ``requirements.txt`` (exists in ``apps/ai-service/requirements.txt``), then ``uvicorn app.main:app --reload --port 8000``.
 - Mobile: standard Flutter commands ``flutter pub get`` and ``flutter run``.
 - Admin dashboard: ``npm install`` and ``npm run dev`` (Vite).

****Operational Considerations****

- Model warm-up at AI service startup to reduce initial latency.
- Monitoring & alerts: add metrics for latency, error rates, model failures (chat/whisper), and emergency escalations.
- Cost control: OpenAI model usage should be monitored; apply sampling or batching where

appropriate.

- Resilience: AI service should degrade gracefully (fallback messages) when OpenAI calls fail (already has fallbacks).

****Files I inspected****

- Backend: backend/api-server/src/app.js
- AI Service entry: apps/ai-service/app/main.py
- AI modules: apps/ai-service/app/services/transcribe.py, apps/ai-service/app/services/emotion.py, apps/ai-service/app/services/risk.py, apps/ai-service/app/services/chat.py
- Mobile: apps/mobile-app/lib/main.dart
- Admin dashboard: apps/admin-dashboard/src/App.tsx

****Open Questions / Next Steps****

- Do you want a PDF/Markdown formatted version of this report saved into the repo?
- Shall I expand with:
- Detailed API reference (endpoints, request/response examples)?
- Security checklist and prioritized remediation tasks?
- Deployment scripts / sample `docker-compose` for local end-to-end testing?
- I can also run a deeper scan of the backend route handlers (controllers) to list specific endpoints and data models if you want.

If you'd like a saved artifact, tell me the format (`markdown` or `pdf`) and filename and I will add it to the workspace.